

AWS LAMBDA

Guia Completa de Referencia

Arquitectura - Seguridad - Observabilidad - CI/CD - Patrones

Mayo 2026

1. Que es AWS Lambda?

AWS Lambda es un servicio de computo serverless que permite ejecutar codigo sin aprovisionar ni administrar servidores. Se paga unicamente por el tiempo de computo consumido: no hay costo cuando el codigo no se esta ejecutando.

Caracteristicas principales:

- **Escalado automatico:** Lambda escala horizontalmente de forma automatica segun la demanda.
- **Stateless:** cada invocacion es independiente; el estado debe persistirse externamente.
- **Event-driven:** se activa en respuesta a eventos de otros servicios AWS o llamadas directas.
- **Pricing por uso:** se factura por numero de invocaciones y GB-segundos de computo.

Ciclo de vida de una invocacion:

Cuando no hay ninguna instancia disponible, Lambda atraviesa las siguientes fases antes de ejecutar tu codigo (cold start):

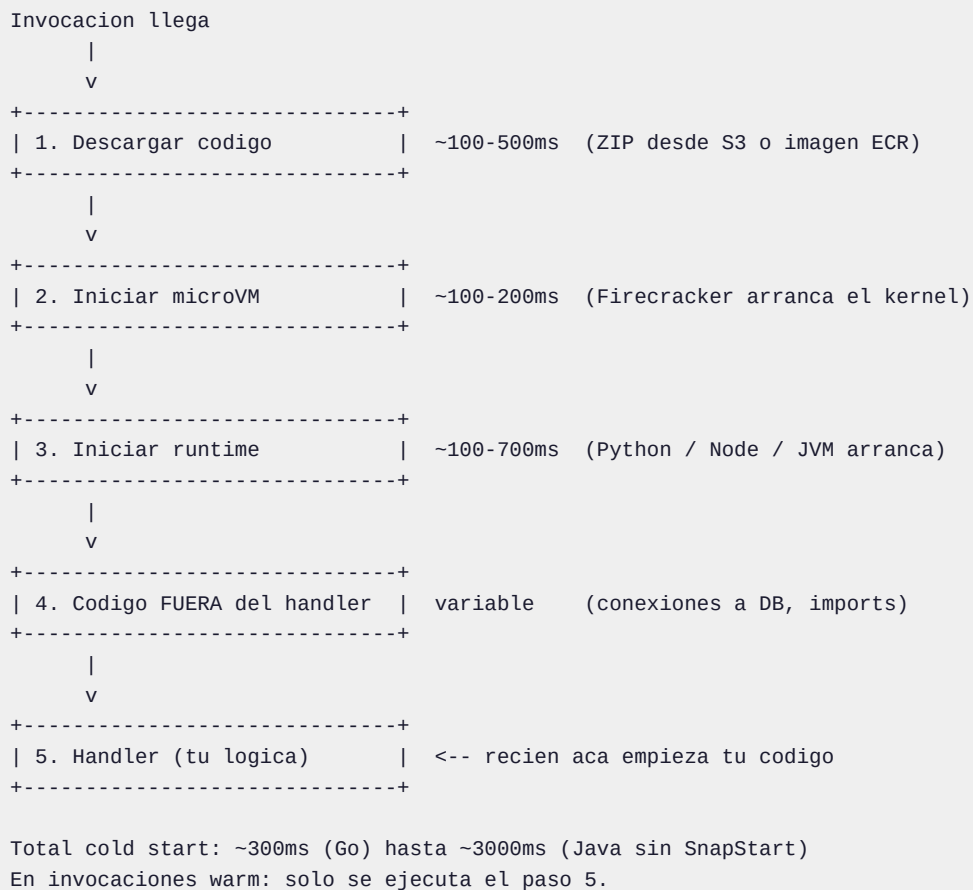


Diagrama de ejecucion warm vs cold:

```
COLD START (primera invocacion o instancia nueva):  
[Descargar] -> [microVM] -> [Runtime] -> [Init code] -> [Handler]  
|<----- demora visible (~300ms a 3s) ----->|  
  
WARM (invocaciones siguientes en la misma instancia):  
                                     [Handler]  
|<--- casi sin demora (~1-5ms overhead) ->|
```

2. Handler y Execution Context

El handler es la funcion de entrada que Lambda invoca. Todo el codigo fuera del handler se ejecuta una sola vez durante el cold start y queda en memoria para invocaciones siguientes (warm start). Esto permite reutilizar conexiones y clientes costosos.

```
import boto3, psycopg2, os

# FUERA del handler: se ejecuta una vez, se reutiliza entre invocaciones
connection = psycopg2.connect(
    host=os.environ["DB_HOST"],
    database=os.environ["DB_NAME"],
)
s3_client = boto3.client("s3") # se crea una sola vez

def handler(event, context):
    # DENTRO del handler: se ejecuta en cada invocacion
    with connection.cursor() as cur:
        cur.execute("SELECT * FROM users WHERE id = %s", (event["userId"],))
        result = cur.fetchone()
    return {"user": result}
```

Que se reutiliza entre invocaciones warm:

Elemento	Se reutiliza?	Notas
Conexiones a DB	Si	Reutilizar siempre que sea posible
Clientes boto3/SDK	Si	Inicializar fuera del handler
Variables globales	Si	Cuidado con estado mutable
Archivos en /tmp	Si	512 MB disponibles por instancia
Variables del handler	No	Se reinician cada invocacion
Objeto context	No	Unico por invocacion
Estado entre microVMs	No	Cada instancia es completamente aislada

Advertencia sobre estado mutable:

```
# PELIGROSO: estado mutable global acumulado entre invocaciones warm
contador = 0

def handler(event, context):
    global contador
    contador += 1 # no es confiable: depende de si la instancia es nueva o no

# CORRECTO: usar estado externo (DynamoDB, Redis, S3)
def handler(event, context):
    dynamodb.update_item(...) # fuente de verdad externa
```

3. Cold Start en Profundidad

Un cold start ocurre cuando Lambda necesita crear una nueva instancia porque no hay ninguna disponible o caliente. Es el principal factor de latencia adicional en arquitecturas serverless.

Cold start aproximado por runtime:

Runtime	Cold Start tipico	Notas
Go / Rust	~100-300ms	Compilados, sin VM, los mas rapidos
Node.js	~300-500ms	Buen balance rendimiento/ecosistema
Python	~300-600ms	El mas usado en data/ML
Java (normal)	~1000-3000ms	JVM es pesada de arrancar
Java SnapStart	~200-400ms	Snapshot pre-inicializado por AWS

Estrategias para reducir el cold start:

- **Provisioned Concurrency:** mantiene instancias pre-calentadas. Elimina el cold start a costo adicional.
- **SnapStart (solo Java):** AWS toma un snapshot del estado post-init. Reduce de ~3s a ~300ms.
- **Reducir el tamaño del paquete:** menos código = menos tiempo de descarga e inicialización.
- **Imports lazy:** importar librerías pesadas solo cuando se necesitan.

```
# MAL: importa toda la libreria al inicio -> aumenta el cold start
import pandas as pd

# MEJOR: importa solo si se necesita en este request
def handler(event, context):
    if event.get("needs_pandas"):
        import pandas as pd    # solo cuando realmente hace falta
```

Provisioned Concurrency — como funciona:

Sin Provisioned Concurrency:
Request -> [cold start: 500ms] -> [handler: 50ms] = 550ms total

Con Provisioned Concurrency (5 instancias pre-calentadas):
Request -> [handler: 50ms] = 50ms total
(la instancia ya estaba lista)

Costo adicional: se paga por las instancias reservadas aunque no reciban tráfico.

4. Paralelismo y Concurrency

Cada instancia de Lambda es un microVM separado usando tecnologia Firecracker de AWS. Esto significa paralelismo real a nivel de infraestructura, no concurrency simulada.

```
Hardware AWS
+-----+
| servidor fisico |
| +-----+ +-----+ +-----+ |
| | microVM | | microVM | | microVM | |
| | [L] 1 | | [L] 2 | | [L] 3 | |
| | proc 1 | | proc 2 | | proc 3 | |
| +-----+ +-----+ +-----+ |
| CPU core A CPU core B CPU core C |
+-----+

Ejecucion verdaderamente paralela

Request 1 -> instancia A (microVM propio) |
Request 2 -> instancia B (microVM propio) | corren en paralelo real
Request 3 -> instancia C (microVM propio) |

Una instancia NO atiende dos requests al mismo tiempo.
Cuando llega un segundo request, AWS levanta una nueva instancia.
```

Limites de concurrency:

Limite	Valor por defecto	Modificable?
Concurrency total/region	1.000 instancias	Si, via AWS Support
Burst limit (us-east-1)	3.000/min	No, fijo por region
Burst limit (otras regiones)	500/min	No, fijo por region
Timeout maximo	15 minutos	No
Memoria maxima	10 GB	No
Payload entrada sincrono	6 MB	No

Que ocurre cuando se alcanza el limite (throttling):

```
Throttling = Lambda rechaza el request porque ya alcanzo su limite.

Invocacion sincrona (API Gateway):
  Cliente -> API Gateway -> Lambda [lleno] -> HTTP 429 Too Many Requests
  El request se pierde si el cliente no reintenta.

Invocacion asincrona (SQS):
  SQS -> Lambda [lleno] -> throttle
  El mensaje vuelve a la cola y reintenta automaticamente.
  No se pierde ningun mensaje.

Por eso SQS como buffer es tan util: absorbe los throttles sin perder eventos.
```

5. Servicios que Pueden Invocar una Lambda

Lambda se integra con practicamente todos los servicios de AWS. Cada servicio entrega un evento con estructura diferente al handler.

5.1 API Gateway — HTTP requests

```
def handler(event, context):
    method = event["httpMethod"]      # GET, POST, etc.
    path   = event["path"]             # /usuarios/123
    body   = json.loads(event.get("body") or "{}")
    params = event.get("queryStringParameters") or {}

    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"mensaje": "ok"})
    }
```

5.2 SQS — mensajes de cola

```
def handler(event, context):
    fallidos = []
    for record in event["Records"]:
        try:
            mensaje = json.loads(record["body"])
            message_id = record["messageId"]
            procesar(mensaje)
        except Exception:
            # Partial Batch Failure: solo este mensaje vuelve a la cola
            fallidos.append({"itemIdentifier": record["messageId"]})

    return {"batchItemFailures": fallidos} # lista vacia = todo OK
```

5.3 SNS — notificaciones pub/sub

```
def handler(event, context):
    for record in event["Records"]:
        sns_msg = record["Sns"]
        subject = sns_msg["Subject"]
        body = json.loads(sns_msg["Message"])
        topic = sns_msg["TopicArn"]
        procesar(subject, body)
```

5.4 S3 — eventos de bucket

```
def handler(event, context):
    for record in event["Records"]:
        bucket = record["s3"]["bucket"]["name"]
        key    = record["s3"]["object"]["key"]
        evento = record["eventName"]    # ObjectCreated:Put, ObjectRemoved, etc.

        if "ObjectCreated" in evento:
            procesar_archivo_nuevo(bucket, key)
```

5.5 EventBridge — eventos programados / cron

```
def handler(event, context):
    source      = event.get("source")          # "aws.events" si es cron
    detail_type = event.get("detail-type")     # "Scheduled Event"
    detail      = event.get("detail", {})      # payload custom si lo definiste
    ejecutar_tarea_periodica()

# serverless.yml
# events:
#   - schedule: cron(0 9 * * ? *)    # todos los dias a las 9 AM UTC
```

5.6 DynamoDB Streams — cambios en tablas

```
def handler(event, context):
    for record in event["Records"]:
        evento = record["eventName"]          # INSERT, MODIFY, REMOVE
        nuevo  = record["dynamodb"].get("NewImage")
        anterior = record["dynamodb"].get("OldImage")

        if evento == "INSERT":    on_creado(nuevo)
        elif evento == "MODIFY":  on_modificado(anterior, nuevo)
        elif evento == "REMOVE":  on_eliminado(anterior)
```

5.7 Cognito — triggers de autenticacion

```
def handler(event, context):
    trigger = event["triggerSource"]    # PostConfirmation_ConfirmSignUp
    username = event["userName"]
    email    = event["request"]["userAttributes"]["email"]
    crear_perfil_en_db(username, email)
    return event    # Cognito requiere devolver el evento (modificado o intacto)
```


6. SQS — Eliminacion de Mensajes

Cuando Lambda consume SQS mediante Event Source Mapping (la forma recomendada), AWS gestiona automaticamente la eliminacion de mensajes. No es necesario eliminarlos manualmente.

Flujo con Event Source Mapping:

```
[SQS] -> Lambda (AWS gestiona la integracion)
|
+-- handler termina sin error -> AWS elimina el mensaje de SQS (OK)
|
+-- handler lanza excepcion    -> mensaje vuelve a la cola y reintenta
                                (hasta MaxReceiveCount veces)
|
+-- supera MaxReceiveCount     -> mensaje va al Dead Letter Queue (DLQ)
```

Si usas polling manual con boto3, SI debes eliminar manualmente:

```
sqs = boto3.client("sqs")

def handler(event, context):
    response = sqs.receive_message(QueueUrl=QUEUE_URL, MaxNumberOfMessages=10)

    for msg in response.get("Messages", []):
        procesar(json.loads(msg["Body"]))

    # Con polling manual: eliminar manualmente
    sqs.delete_message(
        QueueUrl=QUEUE_URL,
        ReceiptHandle=msg["ReceiptHandle"]
    )
```

Partial Batch Failure — solo reencolar los fallidos:

```
Sin Partial Batch Failure (default):
Batch de 10 mensajes, el 3ro falla -> los 10 vuelven a la cola

Con Partial Batch Failure (recomendado):
Batch de 10 mensajes, el 3ro falla -> solo el 3ro vuelve a la cola

Requiere: habilitar "ReportBatchItemFailures" en el Event Source Mapping.
```

7. Formas de Deployar una Lambda

7.1 ZIP directo

```
# 1. Instalar dependencias en carpeta local
pip install -r requirements.txt --target ./package

# 2. Copiar el codigo
cp handler.py ./package/

# 3. Comprimir
cd package && zip -r ../lambda.zip .

# 4. Subir a Lambda
aws lambda update-function-code \
  --function-name mi-funcion \
  --zip-file fileb://lambda.zip

# Limite: 50 MB (zip directo) / 250 MB descomprimido
```

7.2 Lambda Layers — dependencias compartidas

Un Layer es una capa de dependencias reutilizable separada del código. Se puede compartir entre múltiples funciones. Se pueden stackear hasta 5 layers por función.

```
# Crear el layer
mkdir -p layer/python
pip install pandas numpy --target layer/python
cd layer && zip -r ../pandas-layer.zip .

aws lambda publish-layer-version \
  --layer-name pandas-numpy \
  --zip-file fileb://pandas-layer.zip \
  --compatible-runtimes python3.11

# Asociar a una funcion
aws lambda update-function-configuration \
  --function-name mi-funcion \
  --layers arn:aws:lambda:us-east-1:123456789:layer:pandas-numpy:1
```

7.3 Container Image — para dependencias pesadas

```
# Dockerfile
FROM public.ecr.aws/lambda/python:3.11
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY handler.py .
CMD ["handler.handler"]

# Build y push a ECR (Elastic Container Registry)
docker build -t mi-lambda .
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/mi-lambda:latest

# Deployar la funcion desde la imagen
aws lambda update-function-code \
  --function-name mi-funcion \
  --image-uri 123456789.dkr.ecr.us-east-1.amazonaws.com/mi-lambda:latest

# Limite: 10 GB de imagen.
```

7.4 GitHub Actions — CI/CD completo

```

# .github/workflows/deploy.yml
name: Deploy Lambda
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:      ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key:  ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region:            us-east-1

      - id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, tag y push
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG:     ${ github.sha } # cada commit = imagen unica
        run: |
          docker build -t $ECR_REGISTRY/mi-lambda:$IMAGE_TAG .
          docker push     $ECR_REGISTRY/mi-lambda:$IMAGE_TAG

      - name: Update Lambda
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG:     ${ github.sha }
        run: |
          aws lambda update-function-code \
            --function-name mi-funcion \
            --image-uri $ECR_REGISTRY/mi-lambda:$IMAGE_TAG
          aws lambda wait function-updated --function-name mi-funcion

```

Cuadro comparativo de metodos de deploy:

Situacion	Metodo recomendado
Deps livianas (requests, boto3)	ZIP directo o Serverless Framework
Deps compartidas entre funciones	Lambda Layers
Deps pesadas > 250 MB (ML, OpenCV)	Container Image (Docker)
CI/CD profesional	GitHub Actions + ECR + Container Image
boto3 especificamente	No incluirlo: AWS lo provee nativo

8. Variables de Entorno y Credenciales

Regla general: las credenciales nunca viven en GitHub ni hardcodeadas en el código. Hay una separación clara de responsabilidades entre GitHub y AWS.

GitHub Secrets	AWS Secrets Manager / Parameter Store
-----	-----
Solo credenciales para que el pipeline pueda conectarse a AWS	Todo el resto
	- DB_PASSWORD, API_KEY_STRIPE
	- DB_HOST, REDIS_URL
	- feature flags, config por ambiente
v	
AWS_ACCESS_KEY_ID	v
AWS_SECRET_ACCESS_KEY	La Lambda los busca en runtime, nunca en variables de entorno planas.

Donde	Que almacenar	Ejemplo
GitHub Secrets	Solo para conectarse a AWS	AWS_ACCESS_KEY_ID
Secrets Manager	Credenciales sensibles de la app	DB_PASSWORD, API_KEY
Parameter Store	Configuración general	DB_HOST, REDIS_URL
Env vars Lambda	Solo valores no sensibles	AMBIENTE=prod

Acceder a Secrets Manager desde Lambda con cache:

```
import boto3, json, time, os

_cache = {}
CACHE_TTL = 300 # 5 minutos: no llamar a AWS en cada invocación

def get_secret(secret_name):
    now = time.time()
    if secret_name in _cache:
        value, ts = _cache[secret_name]
        if now - ts < CACHE_TTL:
            return value # sirve desde cache, sin latencia extra
    client = boto3.client("secretsmanager")
    response = client.get_secret_value(SecretName=secret_name)
    value = json.loads(response["SecretString"])
    _cache[secret_name] = (value, now)
    return value

# Uso: el ambiente viene de env var no sensible
AMBIENTE = os.environ["AMBIENTE"]
secrets = get_secret(f"{AMBIENTE}/mi-app/db")
```

Anti-patrones a evitar siempre:

- Credenciales hardcodeadas en el código fuente.
- Archivos .env commiteados al repositorio (git history los guarda para siempre).
- Credenciales en variables de entorno de Lambda en texto plano (visibles en la consola).

9. Seguridad

9.1 IAM Roles — principio de minimo privilegio

Cada Lambda tiene un Execution Role: el IAM role que define que puede hacer la funcion dentro de AWS. Nunca usar Action: * o Resource: * en produccion.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes"
      ],
      "Resource": "arn:aws:sqs:us-east-1:123456789:mi-cola-especifica"
      // NO: "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": ["dynamodb:PutItem", "dynamodb:GetItem"],
      "Resource": "arn:aws:dynamodb:us-east-1:123456789:table/mi-tabla"
    }
  ]
}
```

9.2 Lambda dentro de VPC

Por defecto Lambda corre fuera de tu VPC y no puede acceder a recursos privados. Para conectarse a RDS o ElastiCache, se debe configurar la VPC.

```
Sin VPC:
  Lambda -> internet -> APIs publicas   (OK)
  Lambda -> RDS privado                  (NO PUEDE)

Con VPC:
  Lambda -> subnet privada -> RDS       (OK)
  Lambda -> internet (necesita NAT Gateway, tiene costo adicional)

Costo del NAT Gateway: ~$0.045/hora + $0.045/GB procesado.

Nota: Lambda dentro de VPC solia tener cold starts de ~10s.
Desde 2020 AWS lo resolvió con Hyperplane ENIs: ya no es un problema.
```

10. Observabilidad

10.1 Structured Logging

Loggear en formato JSON permite filtrar y consultar logs eficientemente en CloudWatch Logs Insights.

```
import json, logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def handler(event, context):
    logger.info(json.dumps({
        "evento": "orden_procesada",
        "orden_id": event["orden_id"],
        "monto": event["monto"],
        "request_id": context.aws_request_id
    }))

# CloudWatch Logs Insights:
# fields evento, orden_id, monto
# | filter monto > 1000
# | sort @timestamp desc
```

10.2 AWS Lambda Powertools

Librería oficial de AWS que agrega logging estructurado, tracing X-Ray y métricas con mínimo código.

```
from aws_lambda_powertools import Logger, Tracer, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger(service="ordenes")
tracer = Tracer(service="ordenes")
metrics = Metrics(namespace="MiApp", service="ordenes")

@logger.inject_lambda_context    # agrega request_id, funcion, etc.
@tracer.capture_lambda_handler  # traza automáticamente en X-Ray
@metrics.log_metrics            # emite métricas a CloudWatch
def handler(event, context):
    logger.info("Procesando orden", extra={"orden_id": event["id"]})
    metrics.add_metric(name="OrdenesProcessadas",
                      unit=MetricUnit.Count, value=1)
    return procesar(event)
```

10.3 X-Ray Tracing

Permite ver el flujo completo de una request entre servicios, identificando cuellos de botella en la cadena de llamadas.

Ejemplo de traza X-Ray para un request completo:

```
API Gateway -> Lambda A -> DynamoDB
                |
                +-----> Lambda B -> SQS
                        |
                        +-----> RDS
```

Tiempos medidos:

Total:	450ms	
API Gateway:	5ms	
Lambda A:	120ms	
DynamoDB:	15ms	
invoke B:	95ms	
Lambda B:	95ms	
SQS:	10ms	
RDS:	80ms	<-- cuello de botella identificado

11. Modelo de Costos

```
Costo = (invocaciones x $0.0000002)
       + (GB-segundos x $0.0000166667)
```

```
GB-segundo = (memoria en GB) x (duracion en segundos)
```

Ejemplo: 512 MB de RAM, 200ms de duracion, 1 millon de invocaciones/mes

```
Invocaciones: 1.000.000 x $0.0000002 = $0.20
```

```
GB-segundos: 0.5 GB x 0.2s x 1.000.000 = 100.000 GB-s
```

```
100.000 x $0.0000166667 = $1.67
```

```
Total: ~$1.87/mes
```

Free tier mensual: 400.000 GB-s y 1.000.000 invocaciones gratuitos.

Relacion Memoria — CPU asignada:

Memoria	CPU asignada	Notas
128 MB	~0.08 vCPU	Minimo, solo tareas muy livianas
512 MB	~0.32 vCPU	Buen punto de partida
1769 MB	~1.00 vCPU	1 vCPU completo
3538 MB	~2.00 vCPU	Para computo intensivo
10240 MB	~6.00 vCPU	Maximo disponible

Optimizacion con Lambda Power Tuning:

Herramienta open source de AWS (Step Function) que corre la funcion con distintas configs de memoria y genera la curva costo/performance optima. Subir memoria puede ser mas barato si la funcion termina mucho mas rapido:

Funcion CPU-intensiva sin optimizar:

```
128 MB -> 3200ms -> costo: 128 x 3200 = 409.600 unidades
```

```
512 MB -> 800ms -> costo: 512 x 800 = 409.600 unidades (igual precio, 4x mas rapido)
```

```
1024 MB -> 450ms -> costo: 1024 x 450 = 460.800 unidades (5% mas caro, 7x mas rapido)
```

```
2048 MB -> 430ms -> costo: 2048 x 430 = 880.640 unidades (caro, minima mejora)
```

--> En este caso 512 MB es el sweet spot: mismo precio, 4x mas rapido.

12. Step Functions vs Lambda

Lambda ejecuta código. Step Functions decide cuando, en qué orden, cuántas veces y qué hacer si falla ese código. Son complementarias, no competidoras.

El problema sin Step Functions — todo en una Lambda:

```
def handler(event, context):
    validar_stock()      # y si falla aca?
    cobrar_tarjeta()     # se cobro pero no se reservo stock?
    reservar_stock()
    enviar_email()       # y si falla? reintento? double-send?
    notificar_deposito()
    # Si algo falla en el medio -> estado inconsistente
    # Reintentar toda la funcion es peligroso (podria cobrar 2 veces)
```

Con Step Functions — cada paso controlado:

```
orden de compra
  |
  v
[validar_stock]
  |
  v
[cobrar_tarjeta] --falla--> [compensar / reembolsar]
  |
  v
[reservar_stock]
  |
  v
[enviar_email]
  |
  v
[notificar_deposito]
```

Cada [] es una Lambda pequeña y enfocada.
El estado, reintentos y errores los maneja Step Functions.

	Standard	Express
Duración máxima	1 año	5 minutos
Ejecuciones/seg	2.000	100.000+
Precio	Por transición	Por duración + invocaciones
Ideal para	Flujos de negocio	Alto volumen, procesos cortos

Configuración de reintentos con backoff:

```
{
  "Retry": [{
    "ErrorEquals": ["Lambda.ServiceException"],
    "IntervalSeconds": 2,
    "MaxAttempts": 4,
    "BackoffRate": 2.0
  }]
}
```

Tiempos de espera generados:

Intento 1: $2 \times (2^0) = 2$ segundos

Intento 2: $2 \times (2^1) = 4$ segundos

Intento 3: $2 \times (2^2) = 8$ segundos

Intento 4: $2 \times (2^3) = 16$ segundos

13. Backoff Exponencial y Jitter

El backoff exponencial evita bombardear un servicio caído. En vez de reintentar inmediatamente, se espera un tiempo creciente entre intentos.

```
SIN backoff (peligroso):
t=0s [L]x1000 -> todas fallan
t=0s [L]x1000 -> reintentan inmediatamente -> bombardeo al servicio caído
t=0s [L]x1000 -> reintentan inmediatamente -> el servicio no puede recuperarse

CON backoff exponencial:
t=0s [L]x1000 -> todas fallan
t=2s [L]x1000 -> reintentan (espera 2s)
t=6s [L]x1000 -> reintentan (espera 4s)
t=14s [L]x1000 -> reintentan (espera 8s)
Pero todas reintentan al mismo tiempo -> "thundering herd"

CON backoff + JITTER (recomendado):
t=0s [L]x1000 -> todas fallan
t=1.3s [L] -> reintenta
t=1.7s [L] -> reintenta (cada lambda espera distinto)
t=1.9s [L] -> reintenta
...
El servicio puede recuperarse sin picos.
```

Implementacion en Python:

```
import time, random

def llamar_con_backoff(fn, max_intentos=4, base=2, backoff_rate=2):
    for intento in range(max_intentos):
        try:
            return fn()
        except Exception:
            if intento == max_intentos - 1:
                raise # ultimo intento: propagar el error
            espera = base * (backoff_rate ** intento)
            jitter = random.uniform(0, espera) # desincroniza los reintentos
            time.sleep(espera + jitter)

# Uso
resultado = llamar_con_backoff(lambda: llamar_api_externa())
```

14. Comunicacion entre Lambdas

Las Lambdas no pueden llamarse directamente como funciones. Toda comunicacion pasa por un servicio intermediario.

Metodo	Acoplamiento	Espera resultado	Ideal para
Invoke sincrono	Alto	Si	Necesitas la respuesta inmediata
Invoke asincrono	Alto	No	Disparo y olvido simple
SQS	Bajo	No	Procesamiento robusto con reintentos
SNS	Bajo	No	Fan-out: 1 evento a N destinos
EventBridge	Muy bajo	No	Ruteo complejo por reglas
Step Functions	Ninguno	Si	Flujos multi-paso con estado

Invoke sincrono:

```
lambda_client = boto3.client("lambda")

response = lambda_client.invoke(
    FunctionName="lambda-procesar-pago",
    InvocationType="RequestResponse",    # espera resultado
    Payload=json.dumps({"monto": 1500})
)

resultado = json.loads(response["Payload"].read())
# Lambda A espera bloqueada mientras Lambda B ejecuta.
# Si B falla, A recibe el error.
```

Fan-out con SNS:

[illegible]

15. CI/CD Avanzado — Blue/Green y Canary

15.1 Blue/Green con Lambda Aliases

Cada deploy publica una nueva version inmutable. Un alias (ej: 'prod') puede redirigir trafico entre dos versiones, permitiendo rollback instantaneo.

```
Lambda "procesar-orden"
+-- version 1 (codigo viejo) <- BLUE
+-- version 2 (codigo nuevo) <- GREEN
+-- alias "prod"
    +-- 100% -> v1  (antes del deploy)
    +-- 10%  -> v2  (canary, validando)
    +-- 100% -> v2  (luego de validar, deploy completo)

# 1. Publicar nueva version
aws lambda publish-version --function-name procesar-orden

# 2. Canary: 10% al codigo nuevo
aws lambda update-alias \
  --function-name procesar-orden --name prod \
  --routing-config '{"AdditionalVersionWeights": {"2": 0.1}}'

# 3. Si todo OK -> switch completo
aws lambda update-alias \
  --function-name procesar-orden --name prod --function-version 2

# 4. Si algo falla -> rollback instantaneo
aws lambda update-alias \
  --function-name procesar-orden --name prod --function-version 1
```

15.2 Lambda Destinations

Alternativa moderna al DLQ. Define que sucede en exito o fallo en invocaciones asincronas.

```
# serverless.yml
functions:
  procesar:
    handler: handler.handler
    destinations:
      onSuccess: arn:aws:sqs:...:cola-exitos
      onFailure: arn:aws:sqs:...:cola-fallos

# Ventaja sobre DLQ: en exito tambien se puede enrutar,
# y el payload incluye el resultado completo de la funcion.
```

16. Patrones de Arquitectura

16.1 Idempotencia

SQS puede entregar el mismo mensaje mas de una vez (at-least-once delivery). La idempotencia garantiza que procesar el mismo mensaje dos veces produce el mismo resultado.

```
dynamodb = boto3.resource("dynamodb")
tabla = dynamodb.Table("idempotency")

def handler(event, context):
    for record in event["Records"]:
        message_id = record["messageId"]
        try:
            # Si el ID ya existe, la condicion falla -> excepcion
            tabla.put_item(
                Item={"id": message_id, "ttl": int(time.time()) + 86400},
                ConditionExpression="attribute_not_exists(id)"
            )
        except ConditionalCheckFailedException:
            continue # ya procesado, se saltea sin error
        procesar(json.loads(record["body"]))
```

16.2 RDS Proxy — evitar overload en la DB

```
Lambdas (N instancias en paralelo)
[L] -+
[L] -+-> conexiones livianas -> [RDS Proxy] -> 5-20 conexiones reales -> [RDS]
[L] -+ (pool)
[L] -+
```

Sin RDS Proxy: 1.000 lambdas = 1.000 conexiones a la DB (colapso)
Con RDS Proxy: 1.000 lambdas = N conexiones manejadas por el proxy

16.3 Lambda@Edge

Variante de Lambda que corre en los puntos de presencia de CloudFront, cerca del usuario. Ideal para auth, A/B testing, redirects y personalizacion geo.

	Lambda normal	Lambda@Edge (viewer)
Timeout	15 minutos	5 segundos
Memoria maxima	10 GB	128 MB
Env vars	Si	No
VPC	Si	No
Cold start	Normal	Mas frecuente (muchos PoPs)

16.4 SQS como buffer ante picos de trafico

Sin SQS (fragil ante picos):

Requests -> Lambda (puede hacer throttle o sobrecargar la DB)

Con SQS como buffer (robusto):

Requests -> [SQS Queue] -> Lambda (con concurrencia controlada) -> DB

|

+-- absorbe los picos

+-- si Lambda hace throttle, SQS guarda los mensajes

+-- los mensajes nunca se pierden (DLQ como red de seguridad)

17. Testing y Desarrollo Local

17.1 AWS SAM Local

Permite invocar funciones Lambda y levantar APIs localmente sin deployar a AWS.

```
# Instalar SAM CLI
pip install aws-sam-cli

# Invocar funcion con un evento de prueba
sam local invoke MiFuncion --event events/test.json

# Levantar API local (disponible en http://localhost:3000)
sam local start-api

# Generar eventos de prueba para distintos servicios
sam local generate-event sqs receive-message > events/sqs.json
sam local generate-event s3 put > events/s3.json
```

17.2 Localstack

Simula toda la infraestructura AWS en tu maquina con Docker. Permite testear sin gastar en AWS ni necesitar conexion.

```
# docker-compose.yml
services:
  localstack:
    image: localstack/localstack
    ports:
      - "4566:4566"
    environment:
      - SERVICES=lambda,sqs,s3,dynamodb,secretsmanager

# Apuntar boto3 a localstack en lugar de AWS real
sqs = boto3.client(
    "sqs",
    endpoint_url="http://localhost:4566",
    region_name="us-east-1",
    aws_access_key_id="test",
    aws_secret_access_key="test"
)
```

18. Cuando No Usar Lambda

Lambda no es la solución para todo. Hay casos donde otras opciones son más apropiadas.

Caso	Problema con Lambda	Alternativa
Procesos > 15 minutos	Timeout máximo fijo	ECS Fargate
Estado en memoria persistente	Stateless, contexto se recicla	ECS / EC2
GPU / ML intensivo	Sin GPU, max 10 GB RAM	EC2 p3/p4, SageMaker
WebSockets de larga duración	Timeout 29s vía API Gateway	ECS + ALB
Tráfico 100% constante 24/7	ECS puede ser más barato	ECS o EC2 Reserved
Deps muy pesadas > 10 GB	Límite de imagen Docker	ECS con imagen mayor

Resumen y Stack Recomendado para Produccion

Reglas generales:

- Todo lo costoso de inicializar va **fuera del handler** (conexiones, clientes SDK).
- Credenciales sensibles siempre en **Secrets Manager**, nunca hardcodeadas.
- Para overload de DB: usar **RDS Proxy** + concurrencia reservada.
- Para picos de trafico: **SQS como buffer** + Partial Batch Failure.
- Para flujos multi-paso: **Step Functions**, no logica anidada en una Lambda.
- Para observabilidad: **structured logging** + Powertools + X-Ray.
- Para deploy profesional: **GitHub Actions + ECR + Container Image**.
- Para zero-downtime: **alias**es + **canary deployments**.

Stack tecnologico recomendado:

Compute:	Lambda (container image en ECR)
Deploy:	GitHub Actions -> ECR -> Lambda alias
Secrets:	AWS Secrets Manager (cacheado en memoria, TTL 5min)
Config:	AWS Parameter Store
Queue:	SQS + Partial Batch Failure + DLQ
Orquestacion:	Step Functions (Standard para flujos, Express para volumen)
DB proxy:	RDS Proxy (si usas RDS o Aurora)
Cache:	ElastiCache Redis
Observabilidad:	CloudWatch + X-Ray + Lambda Powertools
Testing local:	SAM local o Localstack
IaC:	Serverless Framework o AWS CDK